

FOR OPERATIONS AND FINANCE LEADERS

Before you commission software.

A short, practical guide to deciding whether custom software is the right answer — what it costs, how these projects fail, and the questions worth asking any supplier before you sign anything.

NO SALES PITCH

NINE PAGES

WRITTEN TO BE KEPT

01
The decision
test

02
What it
really costs

03
How projects
fail

04
Questions to
ask suppliers

WHY THIS GUIDE EXISTS / 00

Most bad software projects were decided badly, not built badly.

By the time a build goes wrong, the mistake is usually months old. The scope was vague. Nobody agreed what success looked like. The supplier was chosen on price, or on a conversation that never got specific.

We wrote this because we keep meeting companies partway through that story — and because the decisions that matter most happen **before** anyone writes code, when a supplier has the least incentive to slow you down.

This guide is deliberately useful on its own. If you read it, apply it, and then hire somebody else entirely, it has done its job. There is a page about us at the end; there is nothing about us before it.

HOW TO USE IT

- **Pages 3–4** decide whether to do this at all, and what it should cost.
- **Pages 5–6** are for scoping — read them before you write a brief.
- **Page 7** is a list you can take into any supplier meeting.
- **Page 8** covers the European specifics most guides skip.
- **Page 9** is what a well-run build actually feels like week to week.

WHO IT IS FOR

Operations directors, finance leads and owner-managers at European companies of roughly 25 to 250 people — the size where processes have outgrown spreadsheets but there is no internal product team to hand the problem to.

If you have a CTO and an engineering team, most of this will already be familiar. Page 8 may still be worth ten minutes.

Do you actually need custom software?

Often the honest answer is no. Four questions, in order. If you cannot answer the first one clearly, stop — nothing after it will help.

01 Can you describe the process end to end, out loud, in two minutes?

If three people in your company describe it differently, the problem is not software — it is that the process is undefined. Building software on top of an undefined process encodes the confusion and makes it permanent. **Fix the process on paper first. It is free.**

02 Have you genuinely looked for something off the shelf?

Custom software should be the answer when nothing standard fits, not the first idea. Search properly for a week. If you find something at 80% fit, take it — the missing 20% is almost always cheaper to absorb as a workaround than to build and maintain forever.

03 Is the cost of the current way measurable?

Hours lost, errors made, invoices delayed, customers annoyed. If you cannot put a number on it, you will not be able to tell whether the software worked — and you will struggle to justify the second phase.

04 Will somebody own it after launch?

Software is not a purchase, it is a small ongoing commitment. If no named person will own it, request changes and answer questions, it will drift into disuse within a year regardless of how well it is built.

BUILD IT

The process is **specific to how you compete** — your quoting logic, your routing rules, your compliance workflow. Standard tools force you to work like everyone else, and that is precisely the part you do not want to standardise.

BUY IT

The process is **the same as everyone else's** — payroll, accounting, email, CRM basics, e-signature. Somebody has already built it better than a bespoke project will, and they maintain it for you.

FIX THE PROCESS INSTEAD

The pain comes from **unclear ownership or missing rules**, not from missing tooling. This is the most commonly misdiagnosed case, and the cheapest to resolve.

A USEFUL SIGNAL

If one person is the only one who understands the spreadsheet, you do not have a tooling problem yet — you have a **key-person risk**. That is often the stronger reason to build, and it is easier to get signed off.

A FALSE SIGNAL

“Our competitor has an app.” Not a reason. Build because a process costs you something measurable, not because someone else built something.

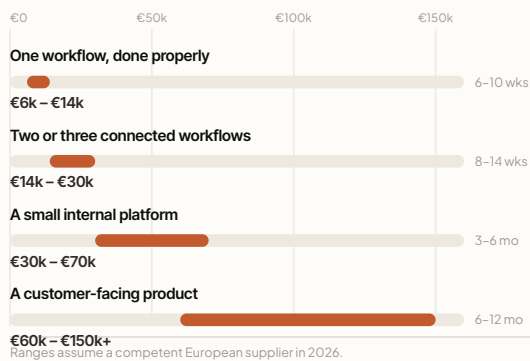
THE SMALLEST USEFUL START

One workflow, end to end, used by real people within six weeks. Broad half-finished systems get abandoned; narrow finished ones get extended.

Honest numbers, and the four things that move them.

Indicative European market ranges for a competent supplier in 2026. Quotes far below these usually mean something has been left out of the scope — most often testing, deployment or handover.

TYPICAL BUILD COST BY SCOPE



AND THE PART NOBODY QUOTES

Budget **15–20% of the build cost per year** for the software to keep working: dependency updates, small changes, hosting, the occasional incident. A supplier who does not mention this is either inexperienced or planning to disappear.

Hosting & infrastructure **€30–200 / mo**

Maintenance & small changes **15–20% / yr**

Your own team's time **2–4 h / week during build**

Training and rollout **1–3 days**

The last line is the one that surprises people. A build with no client time allocated to it will be built against assumptions rather than facts.

THE FOUR THINGS THAT MOVE THE NUMBER / IN ORDER OF IMPACT

01

Integrations

Every existing system you must connect to adds cost and, more importantly, risk you do not control. Two integrations is normal; six is a different project.

02

Number of user roles

Each role brings its own permissions, screens and edge cases. Three roles is not three times one role — it is closer to five.

03

Decision speed

The most expensive variable and the one entirely within your control. A question that waits four days for an answer costs more than any technical choice.

04

Data migration

Moving twelve years of imperfect spreadsheet history is frequently harder than building the software itself. Ask about it early.

If you are choosing between two quotes and one is half the other, the honest question is not "why is that one expensive?" but "what has the cheap one left out?"

HOW THESE PROJECTS FAIL / 03

Five failure modes. All five are preventable before week one.

	WHAT HAPPENS	WHY IT HAPPENS	WHAT PREVENTS IT
01	Scope creeps quietly The build is 80% done for four months.	Nobody wrote down what was in and what was out, so every new idea feels like it was always included.	A written scope with an explicit out-of-scope list , and a change-order process agreed at signature — not invented mid-project.
02	It is built for the wrong person Launch day, and the team refuses to use it.	The specification came from management; the work is done by someone who was never asked.	The people who will actually use it see a working version in week two , not at the end. Their objections are cheap early and expensive late.
03	Nobody can maintain it Six months later, no one can change anything.	Undocumented decisions, an unusual stack, and no handover.	Ask at the outset: who else could take this over? Require a written architecture note and a documented handover as a deliverable, not a favour.
04	It works, but nothing changed The software ships. The process does not improve.	The old way was never switched off, so both run in parallel and the old one wins.	Agree the retirement date for the old process before the build starts, and one metric that proves the change.
05	The supplier disappears Support goes quiet after final payment.	The engagement was priced as a one-off with no relationship after it.	Hold back a final milestone until 30 days after go-live , and make sure you own the code and the accounts from day one.

THE PATTERN UNDERNEATH

Four of the five are failures of **agreement**, not engineering. They are resolved with a paragraph in a document, written before anyone is under pressure. That is why the cheapest hour you will spend on a software project is the one spent writing things down at the beginning.

OWN THESE FROM DAY ONE

- The source-code repository, in your organisation's account.
- The hosting, domain and database accounts, in your name.
- A written architecture note in plain language.
- Admin credentials, held by you and not only by the supplier.

One page is enough. These nine blocks.

You do not need a specification. You need enough clarity that three different suppliers would quote for roughly the same thing — which is the only way their quotes can be compared.

THE ONE-PAGE BRIEF

01 · The process today

Six sentences. Who does what, in what order, using what.

02 · What it costs you now

One number. Hours, errors, delays, or money.

03 · Who uses it

Roles and rough numbers. Name the sceptic.

04 · What must be true at launch

Three to five statements. Testable, not aspirational.

05 · What is explicitly out

The most valuable block on the page.

06 · Systems it must talk to

Names and versions. Say if you are unsure.

07 · Data that must move

Where it lives now, roughly how much, how clean.

08 · Constraints

Budget band, date that matters, data-residency rules.

09 · Who decides

One name. Not a committee.

BLOCK 05 DESERVES ITS OWN NOTE

Most briefs list what the software should do. Almost none list what it should **not** do. That omission is where scope creep is born.

Writing “no mobile app”, “no customer-facing portal”, “no integration with the accounting system in phase one” costs you four lines and saves the project.

WRITE REQUIREMENTS AS OUTCOMES

Instead of
“A dashboard with filters
and export.”

Write
“A depot manager can
see today's late
deliveries in under ten
seconds.”

The second version lets a good supplier propose something better than you imagined. The first turns them into an order-taker.

BEFORE YOU SEND IT OUT

- ☐ Someone who does the work has read it and agreed it is accurate
- ☐ Block 02 contains an actual number
- ☐ Block 05 has at least four items in it
- ☐ Block 09 names one person
- ☐ The whole thing still fits on one page

Twelve questions. Six of them are uncomfortable.

Take this page into the meeting. The answers matter less than whether the supplier is comfortable being asked — and whether they answer in specifics or in adjectives.

ASK	WHAT A GOOD ANSWER SOUNDS LIKE
01 “Walk me through a project like this that went wrong.”	A specific story, what it cost, what changed afterwards. Nobody good has a clean record. A supplier with no failures has either no experience or no candour.
02 “What would you take out of my brief to make this cheaper?”	Two or three concrete cuts and the trade-off for each. Silence here means they will build whatever you say and bill for it.
03 “Who exactly will write this code, and where do they sit?”	Named people, named location. Vagueness here often means it will be quietly subcontracted onward.
04 “What hours will you be available in my timezone?”	A specific window, in writing, in the contract. “We’re flexible” means no overlap.
05 “When will I see something working, not a design?”	Week two or three, on a URL you can open yourself. Anything past week four means you are buying on faith.
06 “Who owns the code and the accounts?”	You do, from day one, in your own organisation. Any other answer is a lock-in you will pay for later.
07 “What happens if you get hit by a bus?”	Documentation, a second person who knows the system, and access you already hold. Ask it bluntly. The answer is revealing.
08 “Which legal entity am I contracting with, and where?”	A company name and a registry number you can look up in a public register. See page 8.
09 “Where will our data physically sit, and who can access it?”	A named region and a named list of people. If they have not thought about this, they have not thought about GDPR either.
10 “What does maintenance cost after launch?”	A percentage or a monthly figure, offered before you ask twice. A supplier who says “nothing” has not run software in production.
11 “How do we handle a change of mind in week four?”	A written change-order process with a rate and a date impact. “We’ll be flexible” is how both sides end up resentful.
12 “What do you need from us for this to go well?”	Specific asks: a decision-maker, weekly access to users, test data. A supplier who needs nothing from you is planning to build in the dark.

Four things worth settling before the contract, not after.

These are the items that stall European software projects in legal review — usually three weeks after everyone thought the deal was done.

01

Which entity, and in which jurisdiction

If your supplier's contracting entity sits outside the EU, a dispute is governed by law you may not want to litigate under. It is a reasonable thing to ask about and a reasonable thing to prefer.

Ask for: the legal name, the registry number, and the country. All three should be verifiable in a public register in under a minute.

02

How the invoice will actually work

An EU-registered, VAT-registered supplier issues a normal intra-community invoice under the reverse-charge mechanism. Your finance team has processed thousands of them. A supplier outside the Union creates a different, unfamiliar piece of paperwork.

Ask for: the VAT number, and a sample invoice before you sign. It is a five-minute check that removes a month of finance friction.

03

Where personal data sits — and who reaches it

The question most suppliers answer incompletely. Hosting in Frankfurt is not the whole answer: **if an engineer outside the EU can open your production database, that access needs to be documented too.** It is entirely workable — it simply has to be written down properly.

Ask for: a data-processing agreement, a sub-processor list, the hosting region, and a straight answer about who can access production data from where.

04

Whether AI is involved, and how it is documented

If any part of the system uses AI, ask what happens to the data you send it, whether the vendor trains on inputs, and what documentation exists. The EU AI Act has made this a governance question rather than a technical curiosity.

Ask for: the model provider, the region it runs in, whether inputs are retained, and a written note of how the feature makes its decisions.

None of this is about preferring European suppliers. It is about knowing which of these four answers you are missing before someone in legal finds out for you.

A SHORT VERSION

Four documents settle almost all of it: the contract, the DPA, the sub-processor list, and a one-page note on data location and access. Any competent supplier can produce all four within a week of being asked.

You should be able to tell by week three.

A well-run six-week build has a recognisable rhythm. If yours does not look roughly like this by week three, raise it then — not at the end.

A SIX-WEEK BUILD, WEEK BY WEEK

WEEK 01

Discovery

Interviews with the people who do the work. The scope document is written and agreed.

Signed scope

WEEK 02

First working version

Something you can open and click. Ugly, incomplete, real.

Alive URL

WEEK 03

The hard part

The genuinely difficult logic. Expect awkward questions this week.

Core logic works

WEEK 04

Roles and edges

Permissions, error states, the cases nobody mentioned in week one.

Usable by all roles

WEEK 05

Real data

Migration, testing with actual records, the things that only break with real input.

Tested with your data

WEEK 06

Handover

Deployment, documentation, training. The old process gets a retirement date.

You own it

Week two is the week that matters. If you have not seen something running by the end of it, ask why before week three.

GOOD SIGNS

- You can open a working URL yourself in week two.
- They ask you awkward questions about edge cases.
- The weekly update mentions something that went wrong.
- They tell you when you are about to make an expensive request.
- Someone who does the work has been interviewed.

WARNING SIGNS

- Every update says “on track” with no detail.
- You have seen designs but nothing running.
- Questions come in batches after long silences.
- Nobody has mentioned testing or handover.
- Scope changes are agreed verbally.

YOUR SIDE OF IT

- Answer questions within 48 hours.
- Send one person to the weekly call, consistently.
- Provide real test data early, not sanitised samples.
- Decide, then hold the decision for two weeks.
- Say when something looks wrong. Immediately.

THE SINGLE BEST PREDICTOR

Whether you can open something and click on it in week two.

Everything else — methodology, stack, credentials, testimonials — is a proxy for this. A supplier who puts working software in front of you early has nothing to hide behind, and a project that has been demonstrated cannot drift far without someone noticing.



ABOUT THE AUTHORS / 08

We wrote this because it is the conversation we have most often.

Nexarrow OÜ is a small Estonian software company. We build internal systems for European businesses — the approvals, records, quoting and reporting workflows that end up living in spreadsheets because no standard tool quite fits.

We are deliberately small. That means we are the wrong choice for anyone who needs a large team or daily availability across a full working day, and a reasonable one for a company that wants a narrow system built properly by people who answer their own email.

WHAT WE BUILD

- Internal operations software
- SaaS product foundations
- AI workflows with EU data residency
- Cloud deployment and reliability

HOW WE WORK

- Fixed scope, milestone-billed
- Working software in week two
- You own the code and accounts
- Documented handover as a deliverable

THE FOUR ANSWERS FROM PAGE 8

Contracting entity	Nexarrow OÜ, Estonia
Registry number	17521430
VAT number	EE102992676
Invoicing	Intra-EU reverse charge
Hosting	EU region by default
DPA & sub-processors	On request
Engineering team	India
Live availability	14:00–18:00 CET

We publish the last two because page 8 says you should ask. It would be odd to write that page and then be vague about our own answer.

IF IT IS USEFUL

We run a paid five-day **Operations Diagnostic**: two interviews, a written map of the process, a ranked list of what is worth automating, and a costed recommendation. You keep the report whether or not we build anything.

hello@nexarrow.eu

nexarrow.eu

Cost ranges are indicative European market figures for 2026 and will vary by country, sector and supplier. Nothing in this guide is legal or tax advice; the items on page 8 are worth raising with your own adviser. Written by Nexarrow OÜ, Tallinn — August 2026. Share it freely.